

WEEK 4 MODULE

ASYMPTOTIC ANALYSIS



Name : Annpurna Sahoo

Roll Number : 2404921530048

Branch : B.Tech CSE (AIML)

Year : 3rd Year

Section : AIML

Subject : Asymptotic analysis

Submission Date : 20.07.26

Week-4 Module

Week 1 module

1. Star patterns

Square -

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Time complexity : $O(n^2)$

- the outer loop runs n times
- for each iteration of the outer loop, the inner loop also runs n times.
- therefore, the total no. of iterations is $n \times n = n^2$.
- hence, the time complexity is $O(n^2)$.

Space complexity : $O(1)$

- the program uses only a few variables (i, j, n).
- No extra array or data structure is used.
- therefore, the space complexity is $O(1)$.

2. Hollow Square

```
* * * * *
*           *
*           *
*           *
*           *
* * * * *
```


T.C - $O(n^2)$

- The outer loop runs n times.
- The inner loop also runs n times for each row.
- Even though only the boundary points * and the inside points spaces, program still checks every position.
- The total no. of iterations is $n \times n = n^2$.
- Hence, time complexity is $O(n^2)$.

Space Complexity: $O(1)$

- The program uses only a few variables (i, j and n).
- No extra array is used.
- Hence, Space complexity is $O(1)$.

3. Triangle

```
*
* *
* * *
* * * *
* * * * *
```

Time Complexity - $O(n^2)$

~~outer loop n times etc~~

- The outer loop runs n times
- The inner loop runs from 1 to i for each row.
- Therefore, the total number of iteration is
 $1 + 2 + 3 + \dots + n = n(n+1)/2$.
- In Big-O notation, lower-order terms & constants are neglected.
- Hence, time complexity is $O(n^2)$.

Space complexity - $O(1)$

- the program uses only a few variables (i, j, n)
- no ~~array~~ extra memory is used.
- therefore, space complexity is $O(1)$.

4. Inverted Triangle

```
*****
****
***
**
*
```

Time complexity - $O(n^2)$

- the outer loop executes n times.
- the inner loop executes $n, n-1, n-2, \dots, 1$ times
- total iterations = $n(n+1)/2$.
- therefore, time complexity is $O(n^2)$.

Space complexity - $O(1)$

- the program uses only a few variables (i, j and n).
- no extra memory is used.
- therefore, space complexity $O(1)$.

5. ~~Pattern~~ Pyramid Pattern - T.C -

- outer loop runs n times.
- inner loops (spaces + stars) together run about $2n-1$ times for each row.
- total operations $\approx n \times (2n-1)$.
- time complexity = $O(n^2)$

Space Complexity -

- only variables (i, j, K, n) are used.
- no extra array is allocated.
- Space complexity = $O(1)$

6. Diamond Pattern -

T.C - $O(n^2)$

- the upper half runs n times.
- the lower half runs $n-1$ times
- In each row, spaces and stars are printed.

S.C - $O(1)$

- only loop variables (i, j, K, n) are used.
- no extra array is used.

7. Half Diamond -

upper half :-

- total iterations = $1 + 2 + 3 + \dots + n = n(n+1)/2$
- complexity = $O(n^2)$

lower half :-

- total iterations = $(n-1) + (n-2) + \dots + 1 = n(n-1)/2$
- complexity = $O(n^2)$

Overall -

$$O(n^2) + O(n^2) = O(n^2)$$

time complexity = $O(n^2)$.

Space Complexity

- NO extra array are used.
- Space complexity = $O(1)$.

8. Half Diamond Inverted Pattern

Time Complexity

- The outer loop runs n times. For each row, the inner loop prints stars equal to the current row no. ($n, n-1, \dots, 1$). Total no. of iterations is -

$$n + (n-1) + (n-2) + \dots + 1 = \frac{n(n+1)}{2}$$

Since $\frac{n(n+1)}{2}$ grows proportionally to n^2 ,

Time complexity is $O(n^2)$.

Space complexity = $O(1)$ because only constant extra memory is used.

Number Pattern

1. Square -

Time Complexity = $O(n^2)$

- The outer loop executes n times.
- For each outer loop iteration, the inner loop also executes n times.
- So total operations = $n \times n = n^2$.
- For $n=5$, the program prints 25 ones.

Space complexity = $O(1)$

1. The program uses only a few variables: i, j and n .

2. No Extra Array is created.

3. Memory usage remains constant regardless of the value of n .

2. Incrementing

→ time complexity :- $O(n^2)$

- the outer loop runs n times (5 rows).
- For each row, the inner loop runs n times (5 columns).
- Total operations = $n \times n$.
- time complexity = $O(n^2)$.

space complexity :- $O(1)$

- the program uses only a few variables (i, j & n).
- no extra array is used.

3. Right Triangle

time complexity :- $O(n^2)$

- outer loop runs n times (for rows).
- Inner loop runs $1, 2, 3, \dots, n$ times
- Total operations = $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$.
- In Big-O notation, this is $O(n^2)$.

Space Complexity :- $O(1)$

- only a few variables (i, j, n) are used.
- No extra array is ~~the~~ required.

4. Inverted Diamond

time complexity :- $O(n^2)$

- outer loop ~~n times~~ runs n times (one time for each row).
- the inner loop prints -
 - Row 1 → n numbers
 - Row 2 → $n-1$ numbers
 - Row 3 → $n-2$ numbers
 - !

! last row $\rightarrow 1$ number
total iterations : $= n + (n-1) + (n-2) + \dots + 1$
 $= \frac{n(n+1)}{2}$

Space complexity : $= O(1)$

- $\rightarrow$ It uses only a few variables (i , j and n)
- $\rightarrow$ No extra array or is used.
- $\rightarrow$ the memory usage remains constant.

5. Sandwich Pattern

Time complexity - $O(n^2)$

- the pattern is printed using nested loops.
- the upper half, middle row, & lower half together print approximately n^2 characters.
- Although the number of characters decreases & then increases, the total no. of operations is still proportional to n^2 .
- time complexity = $O(n^2)$

Space complexity - $O(1)$

- the program uses only a few variables such as i , j and n .
- No extra array is used.
- the memory usage remains constant.

6. Incrementing Diamond Pattern

Time Complexity - $O(n^2)$ because nested loops print both the upper & lower triangular parts of the pattern.

Space Complexity - $O(1)$ because only constant extra memory is used.

Module-2

1. fraud detection - (Linear Search)

Time Complexity -

1) Best Case : $O(1)$

If the suspicious transaction ID is found at the first position, only one comparison is needed.

2) Average Case : $O(n)$

On average, the element is found somewhere in the middle, so about $n/2$ comparisons are performed.

3) Worst Case : $O(n)$

If the transaction ID is at the last position or is not present, the program checks all n elements.

Space Complexity = $O(1)$

- The program uses only a few variables such as n , key & the loop index.
- It does not require any extra memory that depends on the input size.

2. Sequential Search (Binary Search)

Time complexity

Best Case: $O(1)$ -

- the target log ID is found at the middle position in the first comparison.

Average case - $O(\log n)$

- the search space is reduced by half in each step.

Worst case: $O(\log n)$

Even if the element is at the end or not present, Binary Search keeps dividing the search space in half.

Space complexity - $O(1)$ (Iterative Binary Search)

• only a few variables (low, high, mid) are used

→ Recursive Binary Search - $O(\log n)$
Due to recursive function calls on the call stack.

Optimized Approach used - Yes

• The log IDs are already sorted.

• Binary Search is an optimized searching algorithm for sorted data.

• It is faster than linear search because it eliminates half of the search space after every comparison.

3. Sort Employees Salaries (Bubble Sort)

Time Complexity -

Best Case - $O(n^2)$ →

- the program compares adjacent salaries in every pass until the array is sorted.
- therefore, it performs about n^2 comparisons.

Worst Case - $O(n^2)$

If the salaries are in descending order, Bubble Sort performs the maximum number of comparisons & swaps.

Space Complexity = $O(1)$

- the program uses only a temporary variable for swapping salaries.
- No extra array is used.

Optimized Approach Used - No

~~No, because the questions only~~

4. Sort Website Response time (Insertion Time)

Time Complexity -

Best Case : $O(n)$ - when the response times are already sorted.

Average Case : $O(n^2)$

Worst Case : $O(n^2)$ - when the response times are in reverse order.

Space Complexity = $O(1)$

- Insertion Sort sorts the array in place.
- It uses only a few extra variables (Key, i, j), so extra memory remains constant.

Optimized Approach used - Yes

- The problem states that new response times come continuously & must be inserted into the correct position.

- ~~Insertion~~ Insertion Sort is well suited for this situation because it efficiently inserts new elements into an already sorted position of the array.

5. Prioritize Bug Reports (Selection Sort)

Best Case - $O(n^2)$

Average Case - $O(n^2)$

Worst Case - $O(n^2)$

- Selection Sort repeatedly finds the smallest priority score from the unsorted part of the array & places it at the correct position.
- It always performs the same number of comparisons, regardless of whether the array is already sorted or not.
- Time complexity = $O(n^2)$

Space Complexity = $O(1)$

→ Selection Sort sorts the array in place.

→ It uses only a few extra variables (such as minIndex & a temporary variable for swapping).

Module - 3

Topic 1: Merge Sort (Easy)

Ques 1: # Time Complexity: $O(n \log n)$

The list of salary packages is divided into smaller parts until each part contains one element. After that, all parts are merged in sorted order. Dividing takes $\log n$ levels, & merging all package values at each level takes $O(n)$ time. Therefore, the overall time complexity is $O(n \log n)$.

Space Complexity: $O(n)$

Merge Sort uses an additional temporary array while merging the sorted subarrays. Therefore, the ~~overall time~~ extra space required is $O(n)$.

Optimized Approach - Yes

Merge Sort is an efficient divide-and-conquer algorithm. It ~~consistently~~ consistently performs in $O(n \log n)$ time, even in the worst case, which makes it suitable for sorting a large number of placement records.

2. Ques Medium (Merge Sort)

Time Complexity: $O(n \log n)$

The customer order processing ~~at~~ times are repeatedly divided into smaller groups & then merged in sorted order. After sorting, the median is obtained directly; the orders above the median are counted by scanning the list once, & the difference is calculated using the first & last element of the sorted array. Since Merge Sort is the dominant operation, ~~the overall time~~

Complexity
Space complexity: $O(n)$
During the merge step, a temporary array is used to combine the sorted groups. Therefore, the Algorithm requires $O(n)$ extra space.

Topic 2: Quick Sort (Easy)

Time complexity:-

Best case - $O(n \log n)$

Average case - $O(n \log n)$

Worst case - $O(n^2)$

The server response times are sorted by selecting a pivot element & dividing the remaining values into 2 groups. Each group is then sorted using the same process. When the response times are divided into nearly equal groups, the sorting takes $O(n \log n)$ time. If the pivot repeatedly creates uneven groups, the execution time can increase to $O(n^2)$.

Space complexity - $O(\log n)$

The sorting is performed within the original list of server response times. Only the recursive function calls require extra memory, so the average space complexity is $O(\log n)$.

Optimized Approach - Yes

The problem specifies the need for an efficient in-place sorting technique for a large amount of server data. Quick Sort satisfies this requirement because it sorts the response times within the same array & usually provides fast performance for large datasets.

Ques 2. (Medium)

Time Complexity:

Best case - $O(n \log n)$

Average case - $O(n \log n)$

Worst case - $O(n^2)$

The trade values received from different stock exchanges are arranged in descending order using Quick Sort. The algorithm repeatedly places a pivot at its correct position & sorts the remaining values recursively. After sorting, the top 5 trade values are selected, their average is calculated, and the remaining values are checked against the overall average. Since sorting requires the most processing time.

Space Complexity: $O(\log n) \rightarrow$ Average Space Complexity.

The trade values are sorted within the existing array, so no separate array is required. Additional memory is only used for recursive function calls, resulting in an average space complexity of $O(\log n)$.

Topic 3:- Heap Sort (Easy)

Ques 1. # Time Complexity - $O(n \log n)$

The player scores are first organized into a max heap. After that, the highest score is repeatedly moved to its correct position, & the heap is adjusted again until all scores are arranged in ascending order. Building the heap takes $O(n)$ time, while removing & restoring the heap for all elements takes $O(n \log n)$.

Space Complexity - $O(1)$

Heap Sort rearranges the player scores within the same array & does not require an additional array.

for sorting.

Optimized Approach - Yes

The tournament contains a large no. of player scores, so an algorithm with predictable performance is required.

Heap Sort provides $O(n \log n)$ time in all cases while using constant extra memory, making it suitable for generating the final leaderboard efficiently.

Que 2. Medium (Heap Sort)

Time Complexity - $O(n \log n)$

The emergency response times are first converted into a heap. The largest value is then placed at its correct position, & the heap is rebuilt until all response times are arranged in ascending order.

After sorting, the fastest & slowest response times are obtained from the first & last elements, the average is calculated by traversing the list once, & the cases handled faster than the average are counted.

Space Complexity - $O(1)$

Heap Sort performs sorting within the same array of response times. No additional array is required, so the extra space used is $O(1)$.

Optimized Approach - Yes

The hospital needs to process a large no. of emergency records efficiently. Heap Sort provides consistent $O(n \log n)$ performance and uses constant extra memory, making it suitable for generating the required statistics!

Week-4 Module

Problem Statement :-

1. Solⁿ

Code in C++

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int N;
```

```
    cin >> N
```

```
    for (int i=1; i<=N; i++) {
```

```
        cout << i;
```

```
        if (i != N)
```

```
            cout << " ";
```

```
    }
```

```
    return 0;
```

Java Code -

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main (String [] args) {
```

```
        Scanner sc = new
```

```
        Scanner (System.in);
```

```
        int N = sc.nextInt();
```

```
        for (int i=1; i<=N; i++) {
```

```
            System.out.print(i);
```

```
            if (i != N)
```

```
                System.out.print(" ");
```

```
        }
```

```
        sc.close();
```


Algorithm

1. Read the Input Value N .
2. Start a loop from 1 to N .
3. Print the current number.
4. Print a space after every number except the last one.
5. The loop ends after printing all numbers from 1 to N .

Time Complexity :- $O(N)$

The loop runs exactly N times, and each iteration prints one number. Since every iteration takes constant time, the overall time complexity is $O(N)$.

Space Complexity - $O(1)$

The program uses only a few variables & does not allocate any extra memory based on the input size.

More Efficient Approach

No more efficient approach exists because every number from 1 to N must be printed at least once. Hence, $O(N)$ is the optimal time complexity.

Example 1

Input

5

Output

1 2 3 4 5

Explanation -

The no.'s from 1 to 5 are printed in increasing order.

Example 2

Input

8

Output

1 2 3 4 5 6 7 8

Explanation -

The program prints every integer starting from 1 until 8, maintaining increasing order.

onlinegdb.com/online_c++_compiler#

RunDebugStopShareSaveBeautify

Language C++

main.cpp

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int N;
7     cin >> N;
8
9     for(int i = 1; i <= N; i++){
10         cout << i;
11         if(i != N)
12             cout << " ";
13     }
14
15     return 0;
16 }
```

Input

```
5
1 2 3 4 5

...Program finished with exit code 0
Press ENTER to exit console.
```



```
Main.java
1 import java.util.Scanner;
2 public class Main
3 {
4     public static void main(String[] args) {
5         Scanner sc = new
6 Scanner(System.in);
7
8         int N = sc.nextInt();
9         for(int i = 1; i <= N; i++){
10             System.out.print(i);
11             if(i != N){
12                 System.out.print(" ");
13             }
14         }
15         sc.close();
16     }
17 }
18 }
19 }
```



input

8
1 2 3 4 5 6 7 8

...Program finished with exit code 0
Press ENTER to exit console.

2. Problem Statement :

C++ Code -

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int N;
```

```
    cin >> N;
```

```
    long long sum = 0;
```

```
    for (int i = 1; i <= N; i++) {
```

```
        sum += i;
```

```
    }
```

```
    cout << sum;
```

```
    return 0;
```

```
}
```

Java Code -

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main (String[] args) {
```

```
        Scanner sc = new
```

```
        Scanner (System.in);
```

```
        int N = sc.nextInt();
```

```
        long sum = 0;
```

```
        for (int i = 1; i <= N; i++) {
```

```
            sum += i;
```

```
        }
```

```
        System.out.print(sum);
```

```
        sc.close();
```

```
}
```

```
}
```


Time Complexity - $O(N)$

Space Complexity - $O(1)$

- the loop iterates from 1 to N , performing one addition in each iteration. therefore, the running time grows linearly with N .
- only a fixed number of variables (N , sum , and i) are used, regardless of the input size.

More Efficient Approach

Yes, instead of using a loop, the sum can be calculated directly using the mathematical formula:

$$Sum = \frac{N \times (N+1)}{2}$$

this approach reduces the time complexity to $O(1)$ while keeping the space complexity $O(1)$. however, since the problem specifically asks to use a loop, the above solution is appropriate.

Example 1

Input
5

Output
15

Explanation

the sum of the first five natural numbers is:

$$1 + 2 + 3 + 4 + 5 = 15$$

Example 2

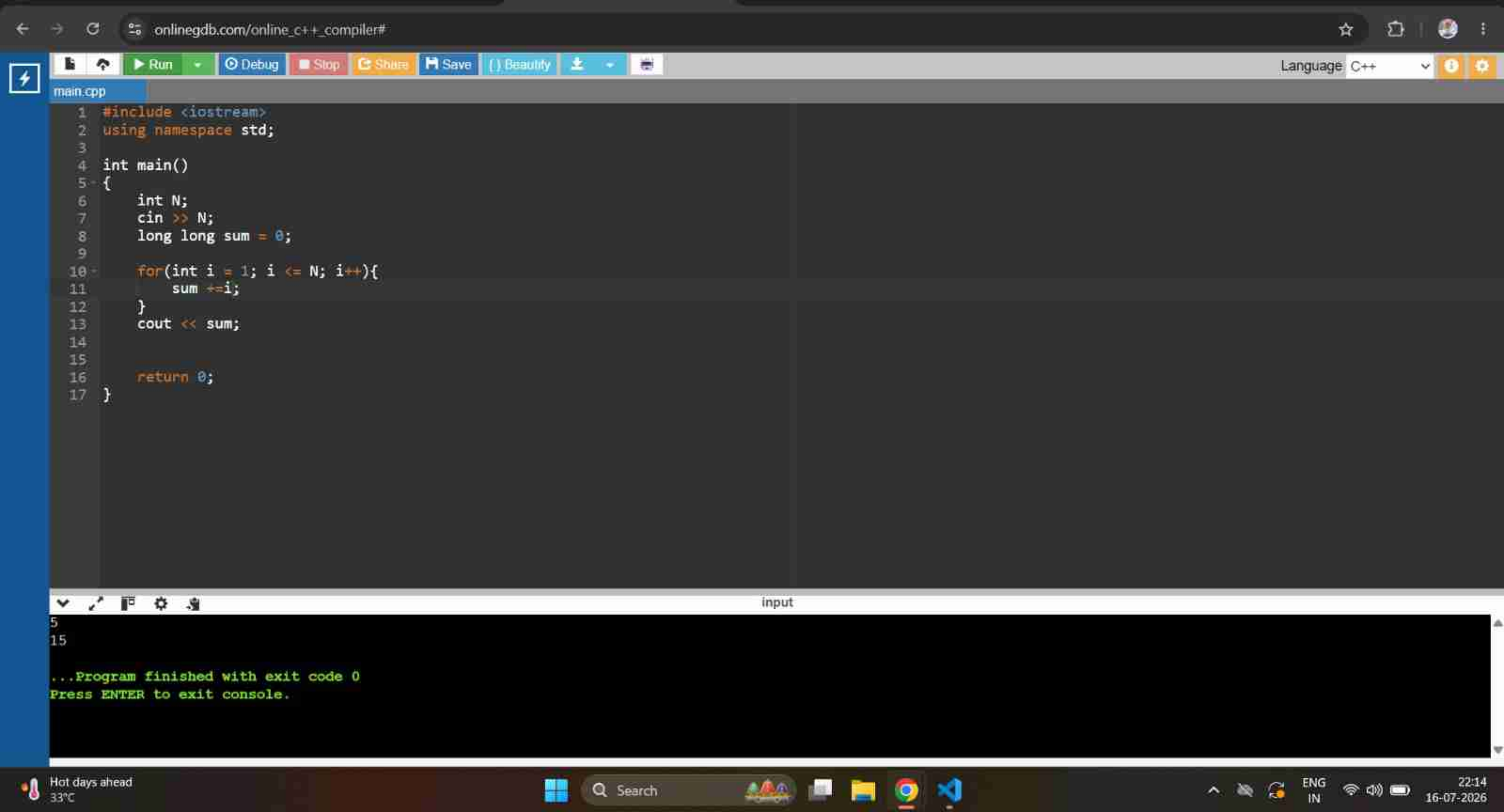
Input
10

Output
55

Explanation

the sum of first ten natural no. is:

$$1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = 55$$




```
Main.java
1- import java.util.Scanner;
2- public class Main{
3-     public static void main(String[] args){
4-         Scanner sc=new
5 Scanner(System.in);
6         int N = sc.nextInt();
7         long sum=0;
8-         for(int i=1; i<=N; i++){
9             sum +=i;
10
11         }
12         System.out.print(sum);
13         sc.close();
14     }
15 }
16
```

input

10
55

...Program finished with exit code 0
Press ENTER to exit console.

Problem Statement :

3 soln

C++ Code -

```
#include <iostream>
using namespace std;
```

```
int main () {
```

```
    int A, B;
```

```
    cin >> A >> B;
```

```
    long long powerValue = 1;
```

```
    for (int i = 0; i < B; i++) {
```

```
        powerValue *= A;
```

```
    }
```

```
    cout << powerValue;
```

```
    return 0;
```

Java Code -

```
import java.util.Scanner;
```

```
public class Main {
```

```
    public static void main (String[] args) {
```

```
        Scanner sc = new
```

```
        Scanner (System.in);
```

```
        int A = sc.nextInt();
```

```
        int B = sc.nextInt();
```

```
        long powerValue = 1;
```

```
        for (int i = 0; i < B; i++) {
```

```
            powerValue *= A;
```

```
        }
```

```
        System.out.print (powerValue);
```

```
        sc.close();
```


Algorithm

1. Start.
2. Read two Integers A and B.
3. Initialize a variable powerValue = 1.
4. Repeat the following steps B times:
 - Multiply powerValue by A.
5. Print powerValue.
6. Stop.

Time Complexity: $O(B)$
The loop executes exactly B times, and each iteration performs one multiplication. $\therefore$ therefore, the running time increases linearly with the value of B.

Space Complexity: $O(1)$
The program uses only a fixed number of variables (A, B, powerValue, and i). No extra memory is allocated based on the input size.

More Efficient Approach -

Yes, A faster method is Binary Exponentiation which computes in $O(\log B)$ time instead of $O(B)$ by repeatedly squaring the base & reducing the exponent.

Example 1

Input

2 5

Output

32

Explanation

→ the value of $2^{<\text{sup}>5^{<|\text{sup}>}}$ is:

$$2 \times 2 \times 2 \times 2 \times 2 = 32$$

Example 2

Input

3 4

output

81

Explanation

→ the value of $3^{<\text{sup}>4^{<|\text{sup}>}}$ is:

$$3 \times 3 \times 3 \times 3 = 81$$

main.cpp

```
1 #include<iostream>
2 using namespace std;
3 int main(){
4     int A,B;
5     cin >> A >> B;
6     long long powerValue =1;
7     for(int i=0; i<B; i++){
8         powerValue *=A;
9     }
10    cout << powerValue;
11    return 0;
12 }
13
```



Input

2 5
32

...Program finished with exit code 0
Press ENTER to exit console.

Main.java

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[] args){
4         Scanner sc=new
5 Scanner(System.in);
6         int A=sc.nextInt();
7         int B=sc.nextInt();
8         long powerValue =1;
9         for(int i=0; i<B; i++){
10             powerValue *=A;
11         }
12 System.out.print(powerValue);
13     sc.close();
14 }
15 }
16 }
```

input

```
3 4
81

...Program finished with exit code 0
Press ENTER to exit console.
```